

Dynamic Extrapolation in Extended Timed Automata

Jensen, Nicolaj Østerby; Jensen, Peter Gjørl; Larsen, Kim Guldstrand

Published in:
Formal Methods and Software Engineering

DOI (link to publication from Publisher):
[10.1007/978-981-99-7584-6_6](https://doi.org/10.1007/978-981-99-7584-6_6)

Creative Commons License
CC BY 4.0

Publication date:
2023

Document Version
Early version, also known as pre-print

[Link to publication from Aalborg University](#)

Citation for published version (APA):
Jensen, N. Ø., Jensen, P. G., & Larsen, K. G. (2023). Dynamic Extrapolation in Extended Timed Automata. In Y. Li, & S. Tahar (Eds.), Formal Methods and Software Engineering: 24th International Conference on Formal Engineering Methods, ICFEM 2023, Brisbane, QLD, Australia, November 21–24, 2023, Proceedings (pp. 83-99). Springer. https://doi.org/10.1007/978-981-99-7584-6_6

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the public portal -

Take down policy

If you believe that this document breaches copyright please contact us at vbn@aub.aau.dk providing details, and we will remove access to the work immediately and investigate your claim.

Dynamic Extrapolation in Extended Timed Automata^{*}

Nicolaj Ø. Jensen, Peter G. Jensen, and Kim G. Larsen

Department of Computer Science, Aalborg University
Selma Lagerlöfs Vej 300, 9220 Aalborg, Denmark
`{noje,pgj,kg1}@cs.aau.dk`

Abstract. Abstractions, such as extrapolation, ensure the termination of timed automata model checking. However, such methods are normally only defined for classical timed automata, whereas modern tools like UPPAAL take as input timed automata extended with discrete data and C-like language constructs (XTA) making classical extrapolation excessively over-approximating if even applicable. In this paper, we propose a new dynamic extrapolation technique for XTAs that utilizes information from the immediate state of the search to find more precise extrapolation values. We determine which code snippets are relevant to obtain the extrapolation values ahead of verification using static analysis and then execute these dynamically during verification. We implement our novel extrapolation technique in UPPAAL and find that it reduces the zone graph sizes by 34.7% overall compared to a classic location-clock-based extrapolation. The best case is an 82.7% reduction and the worst case is a surprising 8.2% increase.

Keywords: Extended timed automata · Extrapolation · Program analysis · Graphs.

1 Introduction

Model checking and verification [2, 9] are powerful tools to ensure the correctness of systems with critical requirements. However, verification of real-time systems relies on abstractions due to the infinite state space of the models. Timed automata, first introduced by R. Alur and D. Dill in [11], is one type of model for time-real systems and it is popular due to the various tools which exist for it today: UPPAAL [10], TCHECKER [18], PAT [15], etc. In timed automata, a countable representation of the state space is achieved using regions, sets of which can be represented as zones, and finiteness is typically achieved with extrapolation. Proposed in [11], a simple form of extrapolation finds a value k such that once the value of a clock x is greater than k , then the exact value of x no longer matter, only that it is greater than k matters. This technique can be extended by finding extrapolation values on a clock- and a location basis as seen in [3].

^{*} Funded by the VILLUM INVESTIGATOR project S4OS.

However, some of the aforementioned tools use more expressive timed automata known as extended timed automata (XTA), where edges are decorated with C-like code and timing constraints can be expressions with integer variables.

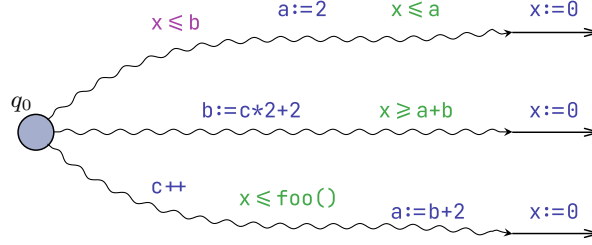


Fig. 1: Extrapolation of clock x in location q_0 depends on all reachable constraints on x between q_0 and resets of x . In XTAs, these constraints may be dynamic and involve expressions.

Consider Fig. 1 showing a fragment of an XTA and paths from location q_0 to resets of clock x . We cannot use typical extrapolation directly since the timing constraints are dynamic and depend on the integer variables a, b, c and the function foo . One solution is to estimate the maximal value of each integer variable and thus each expression. In UPPAAL, the maximal value of variables is computed from type information, which defaults to a large value if not specified by the user. This allows us to find a value k sufficient to achieve a finite verification. However, the lower k is the fewer zones in the zone graph. So we propose a more precise technique. During verification, the exact values of all variables are known in the immediate state of the search. Hence, we can evaluate the expressions to obtain exact extrapolation values. If a timing constraint is not local, we apply the updates along the path to the timing constraint first and then evaluate the expression. For instance, the extrapolation value for clock x in location q_0 in Fig. 1 is the greatest value among the results of evaluating b , evaluating $a := 2$ and then a , evaluating $b := c \cdot 2 + 2$ and then $a + b$, and evaluating $c++$ and then $foo()$. While we are not able to find exact extrapolation values in general due to updates in cycles, we can identify such cases ahead of verification using static analysis. In fact, we can find all relevant updates and expressions ahead of verification, avoiding the overhead of locating relevant constraints whenever extrapolation is needed. When it is not feasible to find exact extrapolation values, we fall back to existing techniques.

Related work. The efficient zone-based abstraction was introduced in [11]. Research has since been exploring ways to ensure the termination of the forward analysis with coarser abstractions and extrapolation techniques. As mentioned, natural extensions include per-clock and per-location extrapolation values [3].

The current implementation of UPPAAL uses LU -extrapolation in the form of the $Extra_{LU}^+$ operator [4] which differentiates upper- and lower bounds as well. The limitations of extrapolation have been investigated in [6, 7]. In [13] the $\alpha_{\leq LU}$ abstraction from [4] is shown to be the biggest abstraction based on LU bounds that is sound and complete for reachability. Additionally, they show how to use non-convex abstractions while working with zones by using the abstractions indirectly. In [12] they use a similar idea for a new simulation relation in order to tackle diagonal constraints, which was shown to be problematic in [6]. See [8] for a survey by P. Bouyer et al. Parametric timed automata (PTA) allow for unknown timing constants to parameterize the model at the cost of undecidability. Recent work [1] investigates several definitions of extrapolation in PTA and finds methods to enhance efficiency and the chance of termination. However, the constant parameters in PTA are not comparable to the discrete variable data in XTA, so the techniques do not transfer. All of the aforementioned work is on timed automata without discrete variable data. The closest related work is therefore in [16] where the authors approximate the ranges of integer variables and thus expressions in XTAs using abstract interpretation. This removes the reliance on users to define accurate ranges, but they are limited to linear expressions. We refine their work by employing a more exact method that partially runs at verification time.

Our contributions. In this paper, we focus on XTAs from UPPAAL [10]. We define a clock- and location-based M -extrapolation for XTAs using an arbitrary domain analysis. We then define the dynamic $DynM$ -extrapolation which gives more accurate extrapolation values in XTAs by taking advantage of the current state. We show how the relevant paths used in $DynM$ -extrapolation can be found ahead of verification and discuss simple extensions which reduce the number of paths, updates, and expressions that need to be evaluated during verification. Finally, we show the effectiveness of our $DynM$ -extrapolation by implementing it in UPPAAL and experimenting on XTA families with dynamic constraints. We find that $DynM$ reduces the zone graph by up to 82.7% compared to M -extrapolation in non-artificial models while being almost on par with M in other cases. Across all experiments, we explore 34.7% fewer zones and use 3.3% less time when using our $DynM$ -extrapolation.

Outline. The outline of the paper is as follows. In Section 2, we recall the definition of XTAs and their semantics. We also define the zone abstraction and M -extrapolation on XTAs. In Section 3, we present our dynamic extrapolation technique and show how it can be broken down to finding acyclic paths in the XTA and then evaluating those paths to find more accurate extrapolation values. We briefly describe extensions to the technique as well. Experiments and results are presented in Section 4, and finally, in Section 5, we conclude on our findings.

2 Preliminary

Let us recall the definition of extended timed automata and its semantics.

2.1 Extended Timed Automata

Clocks and Valuations. Let X be a set of *clock* variables. A *clock valuation* is a function $\nu : X \rightarrow \mathbb{R}_{\geq 0}$ assigning each clock to a non-negative real number. The set $\mathcal{V}(X)$ contains all clocks valuations over X . We use ν_0 to denote the clock valuation such that $\nu_0(x) = 0$ for all $x \in X$. We write $\nu + d$ with $d \in \mathbb{R}_{\geq 0}$ for a clock valuation such that $(\nu + d)(x) = \nu(x) + d$ for all clocks $x \in X$. Lastly, given a set $r \subseteq X$ of clocks to reset, we define $\nu[r]$ such that $\nu[r](x) = 0$ if $x \in r$, and $\nu[r](x) = \nu(x)$ otherwise.

Discrete Data States, Updates, and Expressions. A *discrete data state* (or simply a *data state*) is denoted using σ , and Σ is the set of all data states. An *update* is a function $u : \Sigma \rightarrow \Sigma$ which transforms data states to data states and $\mathcal{U}$ is the set of all updates. We use $e, b : \Sigma \rightarrow \mathbb{Z}$ to denote *expressions* which evaluates to an integer given a data state. Expressions denoted b represent boolean expressions for which evaluation to 0 (false) is of interest. Let E be the set of all expressions. The definition of the discrete data states, updates, and expressions are simplifications of the formalism in UPPAAL. Yet, the definitions provided are sufficient to underpin our method. Our implementation fully supports the imperative language of UPPAAL, including its arrays, functions, and record types.

Constraints. The set $G(X)$ contains clock constraints generated from the grammar $g ::= 1 \mid g_1 \wedge g_2 \mid x \bowtie e$ where $x \in X$, $\bowtie \in \{<, \leq, =, \geq, >\}$, and $e \in E$. If $g \in G(X)$, we write $(x \bowtie e) \in g$ when $x \bowtie e$ is one of the terms of the conjunction g . Given a data state σ , we write $g(\sigma)$ to denote a conjunction similar to g , but where all right-hand sides of the constituent constraints have been evaluated, i.e. each $(x \bowtie e) \in g$ has been replaced by $x \bowtie e(\sigma)$. The satisfaction relation $\models$ is defined naturally for clock valuations and evaluated clock constraints, i.e. $\nu \models g(\sigma)$ iff $\forall (x \bowtie c) \in g(\sigma)$ we have $\nu(x) \bowtie c$.

Definition 1 (Extended Timed Automata (XTA)). An *extended timed automaton* $\mathcal{A}$ is a 6-tuple $(Q, q_0, \sigma_0, T, X, I)$ where

- Q is a finite set of locations,
- $q_0 \in Q$ is the initial location,
- σ_0 is the initial discrete data state,
- $T \subseteq Q \times [G(X) \times E \times \mathcal{U} \times \mathcal{P}(X)] \times Q$ is a set of edges between locations such that if $(q, g, b, u, r, q') \in T$ then q and q' are respectively source and target locations, g is a clock constraint (also called guard), b is a data condition, u is an update, and r is a subset of clocks to be reset,
- X is a non-empty set of clocks, and
- $I : Q \rightarrow G(X)$ assigns clock invariants to locations.

Given any XTA $\mathcal{A}$, we assume that all data states produced by updates in $\mathcal{A}$ fall within a finite domain which we call $\Sigma_{\mathcal{A}}$ with $\sigma_0 \in \Sigma_{\mathcal{A}}$ too. However, we also assume $\Sigma_{\mathcal{A}}$ to be impractical to iterate. It follows that each expression e evaluates to value within a finite domain $\mathcal{D}_{\mathcal{A}}(e)$. We assume that $\mathcal{D}_{\mathcal{A}}(e)$ can be estimated, either from type information or as done in [16].

Definition 2 (Extended Timed Automata Semantics). *The semantics of an XTA $\mathcal{A} = (Q, q_0, \sigma_0, T, X, I)$ is given by a transitions system $\mathcal{S}_{\mathcal{A}} = (S, s_0, \rightarrow)$, where $S = Q \times \Sigma_{\mathcal{A}} \times \mathcal{V}(X)$ is the set of configurations, $s_0 = (q_0, \sigma_0, \nu_0)$ is the initial configuration, and $\rightarrow \subseteq S \times S$ is the transition relation given by:*

$$\frac{\forall 0 \leq d' \leq d : \nu + d' \models I(q)(\sigma)}{(q, \sigma, \nu) \rightarrow (q, \sigma, \nu + d)} \quad \text{where } d \in \mathbb{R}_{\geq 0}, \text{ and} \quad (1)$$

$$\frac{\nu \models g(\sigma) \quad b(\sigma) \neq 0 \quad \nu[r] \models I(q')(u(\sigma))}{(q, \sigma, \nu) \rightarrow (q', u(\sigma), \nu[r])} \quad \text{where } (q, g, b, u, r, q') \in T. \quad (2)$$

In order to verify interesting behavioral properties of an XTA, we must first define zones and symbolic semantics.

2.2 Symbolic Semantics

Future and Reset. Given a set of clocks X , let $W \in \mathcal{P}(\mathcal{V}(X))$ be a set of clock valuations. We define $W^{\nearrow} = \{\nu + d \mid \nu \in W, d \in \mathbb{R}_{\geq 0}\}$ as the future of W . Given a set $r \subseteq X$ of clocks to reset, we define $W[r] = \{\nu[r] \mid \nu \in W\}$ as the reset of r in W .

Zones. Given a set of clocks X , the set of zones $\mathcal{Z}(X)$ is generated from the grammar $Z ::= 1 \mid Z_1 \wedge Z_2 \mid x \bowtie c \mid x - y \bowtie c$, where $x, y \in X$, $c \in \mathbb{N}$, and $\bowtie \in \{<, \leq, =, \geq, >\}$. Given a data state σ , notice that for all $g \in G(X)$ we have $g(\sigma) \in \mathcal{Z}(X)$, and thus $g(\sigma)$ can be considered a zone. The future, reset, and intersection operations all transform zones into other zones.

Difference Bound Matrices. A zone can be represented using a *difference bound matrix* (DBM) and has been the default representation for zones since [11]. We will briefly describe DBMs and point to [5] and [7] for more details. A DBM is a square matrix $D = (c_{i,j}; \prec_{i,j})_{0 \leq i,j \leq n}$ such that $c_{i,j} \in \mathbb{N}$ and $\prec_{i,j} \in \{<, \leq\}$ or $c_{i,j} = \infty$ and $\prec_{i,j} = <$. A DBM D defines a zone $\llbracket D \rrbracket$ which is defined by $\llbracket D \rrbracket = \{\nu \mid \forall 0 \leq i, j \leq n : \nu(x_i) - \nu(x_j) \prec_{i,j} c_{i,j}\}$ where $\{x_1, \dots, x_n\}$ is a set of clocks, and x_0 is a special clock always with the value 0, i.e. $\forall \nu : \nu(x_0) = 0$. A DBM is not a canonical representation of zones, but a normal form can be found by considering it an adjacency matrix of a weighted directed graph and computing the shortest paths closure. DBMs are a useful representation of zones since all operations presented above can be computed efficiently.

Definition 3 (Symbolic Semantics of Extended Timed Automata). *The symbolic semantics of an XTA $\mathcal{A} = (Q, q_0, \sigma_0, T, X, I)$ is given by a transitions system $\mathcal{S}_{\mathcal{A}}^* = (S, s_0, \Rightarrow)$, where $S = Q \times \Sigma_{\mathcal{A}} \times \mathcal{Z}(X)$ is the set of configurations,*

$s_0 = (q_0, \sigma_0, \{\nu_0\})$ is the initial configuration, and $\Rightarrow \subseteq S \times S$ is the transition relation given by:

$$\frac{Z' = (Z^\nearrow \cap I(q)(\sigma)) \quad Z' \neq \emptyset}{(q, \sigma, Z) \Rightarrow (q, \sigma, Z')} \quad , \text{ and} \quad (3)$$

$$\frac{b(\sigma) \neq 0 \quad Z' = (Z \cap g(\sigma))[r] \cap I(q')(u(\sigma)) \quad Z' \neq \emptyset}{(q, \sigma, Z) \Rightarrow (q', u(\sigma), Z')} \quad \text{where } (q, g, b, u, r, q') \in T. \quad (4)$$

The zone abstraction does not guarantee a finite state graph. There may be an infinite number of reachable zones. As suggested in [3], we apply an abstraction $\mathbf{a} : \mathcal{P}(\mathcal{V}(X)) \rightarrow \mathcal{P}(\mathcal{V}(X))$ such that $W \subseteq \mathbf{a}(W)$ and $\mathbf{a}(W) = \mathbf{a}(\mathbf{a}(W))$. The abstract transition system $\Rightarrow_{\mathbf{a}}$ is then given by the induction rule:

$$\frac{(q, \sigma, W) \Rightarrow (q', \sigma', W')}{(q, \sigma, W) \Rightarrow_{\mathbf{a}} (q', \sigma', \mathbf{a}(W'))} \quad \text{where } W = \mathbf{a}(W). \quad (5)$$

A simple way to ensure that the reachability graph induced by $\Rightarrow_{\mathbf{a}}$ is finite is to establish that there is only a finite number of abstractions of sets of valuations. That is, the image of $\mathbf{a}$ is finite. In this case, $\mathbf{a}$ is said to be a *finite abstraction*. Moreover, $\Rightarrow_{\mathbf{a}}$ is said to be sound and complete whenever:

Sound: $(q_0, \sigma_0, \{\nu_0\}) \Rightarrow_{\mathbf{a}}^* (q, \sigma, W)$ implies $\exists \nu \in W : (q_0, \sigma_0, \nu_0) \rightarrow^* (q, \sigma, \nu)$

Complete: $(q_0, \sigma_0, \nu_0) \rightarrow^* (q, \sigma, \nu)$ implies $\exists W : \nu \in W \wedge (q_0, \sigma_0, \{\nu_0\}) \Rightarrow_{\mathbf{a}}^* (q, \sigma, W)$

Completeness follows from the definition of abstraction. Given two abstractions $\mathbf{a}$ and $\mathbf{b}$ such that for any set of clock valuations W we have $\mathbf{a}(W) \subseteq \mathbf{b}(W)$, then we prefer abstraction $\mathbf{b}$ because the graph induced by $\mathbf{b}$ is smaller than the one induced by $\mathbf{a}$.

2.3 M -extrapolation in XTA

In extended timed automata without discrete data, finiteness is achieved with extrapolation based on the constants appearing in the constraints. Modern extrapolation techniques find different extrapolation values per clock and per location for coarser abstractions. Unfortunately, these techniques cannot be used as-is in XTAs, since bounds are expressions instead of constant values. What follows is the location-clock-based M -extrapolation from [3] extended to XTAs based on expression domains.

In XTA $\mathcal{A}$, M is given by:

$$M(q, x) = \max \begin{cases} \max \mathcal{D}_{\mathcal{A}}(e_1) & \text{where } (x \bowtie e_1) \in I(q), \\ \max \mathcal{D}_{\mathcal{A}}(e_2) & \text{where } (q, g, b, u, r, q') \in T \\ & \text{and } (x \bowtie e_2) \in g, \\ M(q', x) & \text{where } (q, g, b, u, r, q') \in T \\ & \text{and } x \notin r \end{cases} \quad (6)$$

By convention $\max \emptyset = -\infty$. The values of M can be found using fixed-point iteration.

Definition 4 (M-equivalence $\equiv_M$ [3]). Given an XTA $\mathcal{A}$ with transition system $\mathcal{S}_{\mathcal{A}} = (S, s_0, \rightarrow)$, then $\equiv_M \subseteq S \times S$ is a relation such that $(q, \sigma, \nu) \equiv_M (q, \sigma, \nu')$ iff for all clocks x either

- $\nu(x) = \nu'(x)$, or
- $\nu(x) > M(q, x)$ and $\nu'(x) > M(q, x)$.

Lemma 1. The relation $\equiv_M$ is a bisimulation relation [3] and therefore preserves reachability, liveness, and deadlock properties.

Definition 5 ($\alpha_{\equiv_M, q, \sigma}$, abstraction w.r.t. $\equiv_M$ [3]). Let W be a set of valuations. Then the abstraction w.r.t. $\equiv_M$ is defined as $\alpha_{\equiv_M, q, \sigma}(W) = \{\nu \mid \exists \nu' \in W : (q, \sigma, \nu) \equiv_M (q, \sigma, \nu')\}$.

Lemma 2. [3] The abstraction $\alpha_{\equiv_M, q, \sigma}$ is sound and complete.

As M -extrapolation uses $\max \mathcal{D}_{\mathcal{A}}$, it is clear that $M(q, x)$ is an over-approximation and not necessarily the tightest bound value relevant for a given clock x in a given location q . It does not take advantage of the data state σ , which is also part of the configuration. In the following section, we will define *dynamic extrapolation*, which uses the given data state σ to evaluate expressions on-the-fly for tighter bound values.

Remark 1. The widely used *LU*-extrapolation technique of [4] can be extended to XTAs in a similar manner as shown above.

3 Dynamic Extrapolation

Fix an XTA $\mathcal{A}$ for this section. Dynamic extrapolation is given by a function $DynM : Q \times \Sigma_{\mathcal{A}} \times X \rightarrow \mathbb{N}$ which takes the discrete data state σ into consideration when determining the extrapolation values of a configuration (q, σ, ν) .

Let us first give a recursive definition of $DynM$:

$$DynM(q, \sigma, x) = \max \begin{cases} e_1(\sigma) & \text{where } (x \bowtie e_1) \in I(q), \\ e_2(\sigma) & \text{where } (q, g, b, u, r, q') \in T \\ & \text{and } (x \bowtie e_2) \in g, \\ DynM(q', u(\sigma), x) & \text{where } (q, g, b, u, r, q') \in T \\ & \text{and } x \notin r \text{ and } b(\sigma) \neq 0 \end{cases} \quad (7)$$

However, unlike for M , it is infeasible to find a fixed-point solution for $DynM$ since, by assumption, it is impractical to iterate all data states $\Sigma_{\mathcal{A}}$. Additionally, some expressions may be expensive to evaluate. We need to deal with the recursiveness in another way.

The definition is recursive whenever there are cycles in the XTA. Trying to determine if cycles can only be repeated a finite number of times is generally undecidable [19]. By assumption of finite domains, it is not undecidable in our

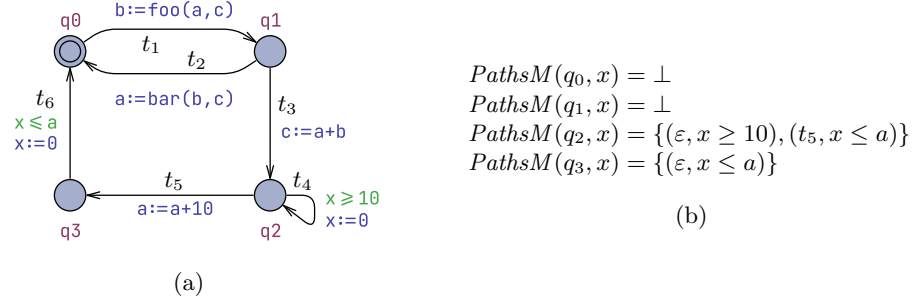


Fig. 2: (a) Example XTA where cycles make *DynM* troublesome to compute. The x is a clock, a, b, c are integer variables comprising the data state, **foo** and **bar** are arbitrary functions. (b) The *PathsM* found by Algorithms 1 and 2.

case, but practically infeasible. Consider the example XTA in Fig. 2a. We can easily compute $\text{DynM}(q_2, \sigma, x)$ and $\text{DynM}(q_3, \sigma, x)$ for any data state σ , but to compute $\text{DynM}(q_0, \sigma, x)$ and $\text{DynM}(q_1, \sigma, x)$ we may have to iterate a huge number of data states due to the cycle consisting of edges t_1 and t_2 . This is the only cycle that is not broken up by a reset of x . We overcome this issue by detecting the problematic cycles and which location-clock pairs (q, x) depend on these cycles. For these cases, we fall back to *M*-extrapolation. In the same algorithm, we also precompute all paths to constraints relevant for each location-clock pair (q, x) , such that they do not have to be found anew each time we want to compute our extrapolation values. Let us introduce some terminology.

A path $w = t_1 \dots t_n = (q_1, g_1, b_1, u_1, r_1, q'_1) \dots (q_n, g_n, b_n, u_n, r_n, q'_n)$ is a sequence of edges from T , such that for all $1 \leq i \leq n-1 : q'_i = q_{i+1}$. We say that the path starts in q_1 and ends in q'_n . A path is cyclic if there are j, k with $j < k$ such that $q_j = q'_k$. A path resets clock x if there exists a j such that $x \in r_j$. We say that $x \bowtie e$ is a local constraint of q if $(x \bowtie e) \in I(q)$ or $(q, g, b, u, r, q') \in T$ such that $(x \bowtie e) \in g$.

Our problem at hand can now be expressed succinctly. Given a location-clock pair (q, x) , we want to find all path-expression pairs (w, e) such that w is a path starting in q that does not reset x , and $x \bowtie e$ is a local constraint at the end of w . Note that w can be empty, in which case the end of w refers to q . For short, we call these *relevant paths* of (q, x) . If any relevant path is cyclic, then we simply need to know that instead, such that we can fall back to *M*-extrapolation. We find all relevant paths for each location-clock pair using a depth-first search (DFS) and coloring. By coloring location-clock pairs as they are visited, the DFS is able to detect cycles. There are four colors with the following meanings:

- White: The location-clock pair is unexplored
- Gray: The location-clock pair is being processed
- Black: All relevant paths have been found for this location-clock pair
- Red: One or more relevant paths starting in this location-clock pair are cyclic

Algorithm 1: ComputePaths

Input : An XTA $(Q, q_0, \sigma_0, T, X, I)$
Output: A mapping $PathsM : Q \times X \rightarrow \mathcal{P}(T^* \times E) \cup \{\perp\}$
 1 Let $PathsM$ be a new map, mapping all location-clock pairs to $\emptyset$;
 2 **foreach** $x \in X$ **do**
 3 Mark all locations as white;
 4 **foreach** $q \in Q$ **do**
 5 **if** q is white **then**
 6 Run ComputePathsRecursive($PathsM, q, x$)
 7 **return** $PathsM$

Algorithm 2: ComputePathsRecursive

Input : A mapping $PathsM : Q \times X \rightarrow \mathcal{P}(T^* \times E) \cup \{\perp\}$, a location q ,
 a clock x
Output: Modifications to $PathsM$
 1 Mark q as gray;
 2 $LC := \{e \mid (x \bowtie e) \in g, (q, g, b, u, r, q') \in T\} \cup \{e \mid (x \bowtie e) \in I(q)\}$;
 3 **foreach** $e \in LC$ **do**
 4 $PathsM(q, x) := PathsM(q, x) \cup \{(\varepsilon, e)\}$
 5 **foreach** $t = (q, g, b, u, r, q') \in T$ **do**
 6 **if** $x \in r$ **then**
 7 Continue
 8 **if** q' is white **then**
 9 Run ComputePathsRecursive($PathsM, q', x$);
 10 // q' is now black or red
 11 **if** q' is red or gray **then**
 12 // We found a cycle
 13 $PathsM(q, x) := \perp$;
 14 Mark q as red;
 15 **return**
 16 **if** q' is black **then**
 17 $PathsM(q, x) := PathsM(q, x) \cup \{(tw, e) \mid (w, e) \in PathsM(q', x)\}$;
 18 Mark q as black;
 19 **return**

The algorithm constructs a mapping $PathsM : Q \times X \rightarrow \mathcal{P}(T^* \times E) \cup \{\perp\}$ that assigns each location-clock pair to a finite set of relevant path-expression pairs or $\perp$ if the location-clock pair has a cyclic relevant path. The algorithm for finding $PathsM$ is shown in Algorithms 1 and 2. Fig. 2b shows the resulting $PathsM$ when the algorithm is run on the XTA in Fig. 2a.

Lemma 3. *Let $PathsM$ be the result of Algorithms 1 and 2. If $PathsM(q, x) \neq \perp$ then $PathsM(q, x)$ contains all relevant paths (w, e) such that w is acyclic,*

does not reset x , starts in q , and ends in q' such that $x \bowtie e$ is a local constraint to q' .

Lemma 3 is easily proven by showing that the loop invariants induced by the definition of the colors hold at the exit of Algorithm 2, and therefore we do not include the proof in this version of the paper.

We can now define $PDynM$ which approximates $DynM$ by using $PathsM$ and auxiliary function $eval$:

$$PDynM(q, \sigma, x) = \begin{cases} M(q, x) & \text{if } PathsM(q, x) = \perp, \\ \max_{(w, e) \in PathsM(q, x)} eval(w, e, \sigma) & \text{otherwise} \end{cases} \quad (8)$$

$$eval(tw, e, \sigma) = \begin{cases} eval(w, e, u(\sigma)) & \text{if } t = (q, g, b, u, r, q') \wedge b(\sigma) \neq 0, \\ -\infty & \text{otherwise} \end{cases} \quad (9)$$

$$eval(\varepsilon, e, \sigma) = e(\sigma) \quad (10)$$

This definition is well-defined and feasibly computable by using $M(q, x)$ as a fallback whenever $PathsM(q, x) = \perp$, i.e. in locations with cyclic paths to relevant constraints. In locations without cycles, $PDynM$ is equivalent to the ideal $DynM$. We shall now argue, that $PDynM$ is a valid abstraction. Specifically:

Definition 6 (PDynM-equivalence $\equiv_{PDynM}$). *Given an XTA $\mathcal{A}$ with transition system $\mathcal{S}_{\mathcal{A}} = (S, s_0, \rightarrow)$, then $\equiv_{PDynM} \subseteq S \times S$ is a relation such that $(q, \sigma, \nu) \equiv_{PDynM} (q, \sigma, \nu')$ iff for each clocks x either*

- $\nu(x) = \nu'(x)$, or
- $\nu(x) > PDynM(q, \sigma, x)$ and $\nu'(x) > PDynM(q, \sigma, x)$.

Lemma 4. *The relation $\equiv_{PDynM}$ is a bisimulation relation and therefore preserves reachability, liveness, and deadlock properties.*

Proof. From Lemma 3 and the definition of $PDynM$, it is clear $PDynM(q, \sigma, x)$ is either equal to $M(q, x)$ or is the greatest value obtained from evaluating the updates and data conditions along a relevant path of (q, x) and then a final bound expression, local to the end of the path. Hence, by Lemma 3, $PDynM(q, \sigma, x)$ must be equal to or greater than the greatest value which x is ever compared to before getting reset in any path starting from q . This implies that ν and ν' satisfy the same clock constraints and it follows that $\equiv_{PDynM}$ a bisimulation relation. $\square$

Definition 7 ($\mathbf{a}_{\equiv_{PDynM}}$, abstraction w.r.t. $\equiv_{PDynM}$). *Let W be a set of valuations. Then the abstraction w.r.t $\equiv_{PDynM}$ is defined as $\mathbf{a}_{\equiv_{PDynM}, q, \sigma}(W) = \{\nu \mid \exists \nu' \in W : (q, \sigma, \nu) \equiv_{PDynM} (q, \sigma, \nu')\}$.*

Lemma 5. *The abstraction $\mathbf{a}_{\equiv_{PDynM}, q, \sigma}$ is sound and complete, and it is coarser than $\mathbf{a}_{\equiv_{M, q, \sigma}}$*

Proof. Completeness is obvious. Soundness follows from Lemma 4. The increased coarseness follows from the definitions, where we have that $PDynM(q, \sigma, x) \leq M(q, x)$ for any location q and clock x . $\square$

The abstraction $\mathbf{a}_{\equiv_{PDynM}, q, \sigma}$ does not preserve convexity of valuation sets. Even if Z is a convex zone, $\mathbf{a}_{\equiv_{PDynM}, q, \sigma}(Z)$ is not always convex, which means it cannot be represented with a DBM. We shall therefore define a location- and data-dependent extrapolator called $Extra_{PDynM, q, \sigma}^+$ which is an operator defined on DBMs and approximates $\mathbf{a}_{\equiv_{PDynM}, q, \sigma}$. The naming and the definition are intentionally similar to that of $Extra_M^+$ in [4] since the only difference is that our extrapolation values come from $PDynM$ instead of M . If D is a DBM in normal form given by $(c_{i,j}; \prec_{i,j})_{0 \leq i, j \leq n}$, then $Extra_{PDynM, q, \sigma}^+(D)$ is defined as:

$$(c'_{i,j}; \prec'_{i,j}) = \begin{cases} (\infty; <) & \text{if } c_{i,j} > PDynM(q, \sigma, x_i) \\ (\infty; <) & \text{if } -c_{0,i} > PDynM(q, \sigma, x_i) \\ (\infty; <) & \text{if } -c_{0,j} > PDynM(q, \sigma, x_j), i \neq 0 \\ (-PDynM(q, \sigma, x_j); <) & \text{if } -c_{i,j} > PDynM(q, \sigma, x_j), i = 0 \\ (c_{i,j}; \prec_{i,j}) & \text{otherwise} \end{cases} \quad (11)$$

Note that $Extra_{PDynM, q, \sigma}^+(D)$ is not necessarily in normal form. From the extrapolator $Extra_{PDynM, q, \sigma}^+$ follows the abstraction $\mathbf{a}_{Extra_{PDynM, q, \sigma}^+}$.

Theorem 1. *The abstraction $\mathbf{a}_{Extra_{PDynM, q, \sigma}^+}$ is sound, complete, and preserves zone convexity.*

Proof. The extrapolator $Extra_{PDynM, q, \sigma}^+$ transforms zones into larger zones, which implies completeness and zone convexity. Finiteness follows from the fact that all the DBM coefficients fall within a finite number of values. Soundness is not obvious, and we point to [4] where a similar abstraction is proven sound. $\square$

3.1 Reducing Relevant Paths

Let us briefly describe ways to reduce the number of relevant paths.

Strongly Connected Components. So far, we have deemed all cycles as problematic based on the fact that their updates can be repeated indefinitely resulting in an infeasible number of possible data states to evaluate our constraint expression in. However, this issue does not apply to cycles where all updates are identity updates, i.e. updates u_{id} such that $u_{id}(\sigma) = \sigma$ for all $\sigma \in \Sigma_A$. Repeating u_{id} indefinitely makes no difference. Hence, we can reduce the number of cases where $PathsM(q, x) = \perp$ by essentially computing strongly connected components (SCCs) in the XTA. The internal edges of SCCs must have empty data conditions and identity updates, and cannot reset x . The latter condition implies that we must compute an SCC graph for each clock $x \in X$, but since SCCs can be constructed in linear time using Tarjan's algorithm, this is not a concern. Using the SCC graphs, we can compute $PathsM_{SCC}$ which assigns $\perp$ to fewer cases than $PathsM$. Remark that cycles of identity updates are often modeling mistakes and the models used in the experiments in Section 4 do not contain any such cycles.

Redundant Paths Using Expression Domains. Let $(\mathcal{P}(\mathbb{Z}), \sqsubseteq)$ be a partial order such that if $A, B \subseteq \mathbb{Z}$ then $A \sqsubseteq B$ iff $\forall a \in A. \forall b \in B : a \leq b$. We can use this to determine which bound programs will never be the greatest relevant bound expression. If a bound expression is never the greatest relevant bound, then we can discard the path leading to it, reducing the number of paths that needs to be evaluated. The function $PathsM^*$ below is a subset of $PathsM$ without these redundant paths:

$$\begin{aligned}
PathsM^*(q, x) = & \\
& \begin{cases} \perp & \text{if } PathsM(q, x) = \perp \\ \{(w, e) \in PathsM(q, x) \mid \neg \exists (w', e') \in PathsM(q, x) : \mathcal{D}_A(e) \sqsubseteq \mathcal{D}_A(e')\} & \text{otherwise} \end{cases}
\end{aligned} \tag{12}$$

The more precise (i.e. smaller) the domains of $\mathcal{D}_A$ are, the better this improvement is.

3.2 Dynamic LU-extrapolation

We can easily extend the dynamic extrapolation in a similar manner as proposed in [4] and achieve a lower- and upper-bound aware extrapolation. We simply distinguish between bounds using $\triangleright \in \{>, \geq, =\}$ and $\triangleleft \in \{<, \leq, =\}$ in line 2 of `ComputePathsRecursive` in Algorithm 2 in order to compute separate $PathsL$ and $PathsU$, which are mappings to paths respectively leading to lower- and upper-bound expressions. This allows us to create an abstraction $\mathbf{a}_{\preceq_{PDynLU, q, \sigma}}$ based on the simulation relation $\preceq_{PDynLU, q, \sigma}$, which is correct for reachability, but not for deadlock and liveness properties. While orthogonal techniques, the resulting $PDynLU$ -extrapolation is only slightly coarser than LU -extrapolation in practice, which is why we have not defined $PDynLU$ in detail.

3.3 A Note on Timed Automata Networks

In tools for modeling timed systems using timed automata, e.g. UPPAAL [10], it is common to have timed automata *networks* consisting of multiple timed automata components. The dynamic extrapolation technique presented in this paper extends to this formalism with some caveats. $PathsM$ and $PDynM$ can be found and evaluated on a component-wise basis such that for a clock x the greatest bound across all components is used, similar to M -extrapolation in [3]. This is correct for shared clocks too. Remark, however, that dynamic extrapolation cannot be done component-wise in networks with expressions relying on a globally shared data state. In networks, a relevant path of one component may be interleaved with transitions in other components. If the other components can change the globally shared state, then our assumption that the relevant path contains all updates leading up to the evaluation of the guard breaks down. If we instead attempt to find relevant paths in the composed network as opposed to the component-wise approach, the number of relevant paths increases exponentially, resulting in exponentially more work during verification whenever we extrapolate.

4 Experiments and Results

In order to test the contributions of this paper, we have implemented a prototype of our dynamic extrapolation technique in UPPAAL [10]. We use the SCC improvement and discard redundant paths as discussed in Section 3.1 and compute $PathsM_{SCC}^*$ once prior to verification. During verification, the paths are used to find the extrapolation values for each clock whenever a configuration is explored. We will refer to our implementation simply as *DynM* in this section.

In the experiments, various models are used. Some of the models have dynamic constraints using expressions, others have not. Our experiments show that our dynamic extrapolation never reduces the size of the zone graph compared to its non-dynamic counterpart M unless there is at least one dynamic constraint or unless data conditions can exclude bound expressions. We shall therefore delimit our results to all models in our repository with dynamic constraints such that dynamic extrapolation may take effect. These models include:¹

- Simple (Simple- I): The artificial example from [16] designed to showcase the issues of relying purely on type information for extrapolation in XTA.
- TCP back-off protocol (TCP- V - N): In this model, N clients estimate a congestion window based on how often their messages to the server are lost/denied. They limit their rate of messaging inversely proportional to the size of the congestion window. Our experiments include both an additive-increase/multiplicative-decrease (AIMD) back-off protocol and a linear back-off protocol.
- Gossip Protocol (Gossip- V - N): In this classical model of information sharing, N components each have a secret. They can call each other and exchange their secrets. We use two variants of this model. In one variant, the duration of the call between two components is the union of their known secrets. In the other, the duration is the size of the symmetric difference of their known secrets plus one.
- Firefly synchronization (Firefly- W - H - N): In this model, N fireflies live on an W by H grid. A firefly blinks every $60 - t$ seconds, where t is the number of times it has seen another firefly in the same cell blink at least 30 seconds after its own latest blink. Over time this results in the fireflies blinking in synchronization.
- Leader election (Leader- N): In this model, N network nodes attempt to elect a leader by communicating with each other. The timeout deadline is dynamically calculated based on the number of hops a message do.
- Printing projects (Printing- M - P): In this model, P projects require a varying number of pieces printed. Each piece has a different size and will therefore take a different amount of time to print. There are M printers available.

All integer variables in the models use the default integer range from -32768 to 32767 . There is no globally shared state. Each model is run in UPPAAL using

¹ The test models and results can be found at
<https://github.com/NicEastvillage/DynamicExtrapolationDataArtifact>

Model	<i>M</i>		<i>DynM</i>		<i>DynM/M%</i>	
	Zones	Time	Zones	Time	Zones	Time
Simple-7	32,772	14,589	4	0	0.01%	0.00%
Simple-100	32,772	14,440	4	0	0.01%	0.00%
Simple-1000	32,772	14,675	4	1	0.01%	0.01%
Firefly-1-1-10	6,501	176	6,501	255	100.00%	144.89%
Firefly-1-1-15	49,233	179,011	49,233	180,645	100.00%	100.91%
Firefly-1-1-20	-	-	-	-	-	-
Firefly-2-1-4	167,359	25,987	167,359	27,345	100.00%	105.23%
Firefly-2-1-5	-	-	-	-	-	-
Gossip-symdiff-3	135	3	135	8	100.00%	266.67%
Gossip-symdiff-4	13,792	538	6,972	356	50.55%	66.17%
Gossip-symdiff-5	4,699,522	986,726	919,243	144,123	19.56%	14.61%
Gossip-symdiff-6	-	-	-	-	-	-
Gossip-union-3	138	3	138	3	100.00%	100.00%
Gossip-union-4	16,536	99	17,640	133	106.68%	134.34%
Gossip-union-5	3,016,463	25,906	3,264,863	35,047	108.23%	135.29%
Gossip-union-6	-	-	-	-	-	-
Leader-2	40	0	40	0	100.00%	-
Leader-3	127,537	1,070	127,537	1,271	100.00%	118.79%
Leader-4	-	-	-	-	-	-
Printing-2-5	7,037	105	7,257	145	103.13%	138.10%
Printing-2-6	121,135	3,064	125,525	4,484	103.62%	146.34%
Printing-2-7	1,546,909	78,747	1,601,757	108,729	103.55%	138.07%
Printing-2-8	49,755,991	7,914,155	51,128,387	9,448,237	102.76%	119.38%
Printing-2-9	-	-	-	-	-	-
TCP-aimd-2	66,261	385	28,736	218	43.37%	56.62%
TCP-aimd-3	37,528,973	807,340	6,486,638	87,975	17.28%	10.90%
TCP-aimd-4	-	-	-	-	-	-
TCP-linear-2	75,045	307	62,358	389	83.09%	126.71%
TCP-linear-3	46,989,547	813,729	30,274,603	479,942	64.43%	58.98%
TCP-linear-4	-	-	-	-	-	-
SUM	144,286,460	10,881,055	94,274,934	10,519,305		
% of M	100%	100%	65.34%	96.68%		

Table 1: Number of zones in the zone graph and time spent exploring the zone graph when using *M*- and *DynM*-extrapolation in various models with dynamic constraints. The better results are highlighted in bold. A dash '-' indicates that the experiment did not finish within the allocated 4 hours. Time is in milliseconds.

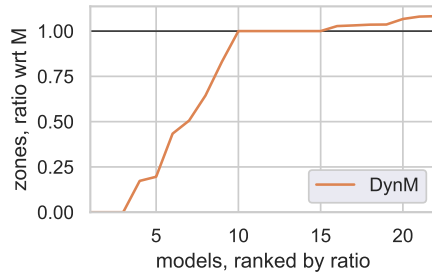


Fig. 3: Cactus plot of the number of zones in the zone graph as a ratio w.r.t. *M*-extrapolation

M - and $DynM$ -extrapolation with a time limit of 4 hours, and we measure the size of the zone graph and the time spent exploring it for each model. Many properties of these models can be verified without visiting the entire zone graph, but these metrics can describe the relative coarseness and overhead of the two extrapolation techniques and give insight into their performance in a worst-case scenario. Our results can be seen in Table 1. We find that $DynM$ - outperforms M -extrapolation in some families of models while being almost on par with M in others. Whenever there are improvements, those improvements are exponentially better as the parameters of the models are increased. Especially the constructed Simple- I model shows the potential of $DynM$ -extrapolation. The abstract interpretation technique in [16] results in a number of zones proportional to the parameter I , while we always have 4 zones regardless of I . However, we also observe a time overhead of up to 46.3% in the worst case. The time overhead is not surprising given that $DynM$ is evaluating additional expressions during verification compared to M . There are two anomalies in our results. Specifically, we have that M -extrapolation produces fewer zones in the printing-project models and in the gossip-protocol models using union despite our Lemma 5 stating that $DynM$ is a coarser abstraction. Having confirmed that $DynM$ finds equal or smaller extrapolation values for these models, we hypothesize these anomalies occur due to how UPPAAL merges zones, and this merging happens to benefit the experiments using M in these models.

Summed across all models, $DynM$ -extrapolation results in 65.3% zones and 96.7% time spent compared to M -extrapolation, a reduction of 34.6% and 3.3%, respectively. Despite anomalies and occasional increased time overhead, the $DynM$ -extrapolation provides great memory savings in the general case, at least on the given benchmark set. Since memory is usually a more limited resource, this is a satisfactory outcome.

Finally, Fig. 3 is a plot of the ratios of the zone graph sizes w.r.t. M -extrapolation, where the test instances have been sorted along the x-axis based on their ratio. The graph clearly shows that $DynM$ -extrapolation notably outperforms on a subset of the models, while also maintaining comparable performance on other test models. When using $DynM$ -extrapolation, the number of zones in the zone graph of TCP-aimd-3 is 17.3% of what it is when using M -extrapolation, which is the greatest reduction among our experiments, if we do not consider the artificially constructed Simple model, where the reduction is close to 100%.

5 Conclusion

In this paper, we refine extrapolation techniques for extended timed automata (XTA), which is the input used in practice in tools like UPPAAL. Our technique takes advantage of the immediate state of the search to evaluate more exact extrapolation values during verification. Our experiments show that our dynamic $DynM$ -extrapolation is a better abstraction for some families of XTA while being almost on par with its non-dynamic counterpart in other cases. Our summed

reduction in the sizes of zone graphs is 34.7%. The best reduction of a zone graph is by 82.7% and our worst case is an increase of 8.2%. Our technique has a high time overhead, up to 46.3% slower in the worst case, but the overall time spent is reduced by 3.3%.

Future Work. Since the paths of *PathsM* are essentially programs, possible optimizations include dead-code elimination and constant propagation [14, 17]. We also want to use a live variable analysis to further reduce the number of cases where $\text{PathsM}(q, x) = \perp$.

References

1. Arcile, J., André, É.: Zone extrapolations in parametric timed automata. In: *Lecture Notes in Computer Science*, pp. 451–469. Springer International Publishing (2022). https://doi.org/10.1007/978-3-031-06773-0_24
2. Baier, C., Katoen, J.P.: *Principles of Model Checking*. The MIT Press (2008)
3. Behrmann, G., Bouyer, P., Fleury, E., Larsen, K.G.: Static guard analysis in timed automata verification. In: Garavel, H., Hatcliff, J. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems*. pp. 254–270. Springer Berlin Heidelberg, Berlin, Heidelberg (2003). https://doi.org/10.1007/3-540-36577-X_18
4. Behrmann, G., Bouyer, P., Larsen, K.G., Pelánek, R.: Lower and upper bounds in zone based abstractions of timed automata. In: Jensen, K., Podelski, A. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems*. pp. 312–326. Springer Berlin Heidelberg, Berlin, Heidelberg (2004). <https://doi.org/10.1007/s10009-005-0190-0>
5. Bengtsson, J., Yi, W.: *Timed Automata: Semantics, Algorithms and Tools*, pp. 87–124. Springer Berlin Heidelberg, Berlin, Heidelberg (2004). https://doi.org/10.1007/978-3-540-27755-2_3
6. Bouyer, P.: Untameable timed automata! In: Alt, H., Habib, M. (eds.) *STACS 2003*. pp. 620–631. Springer Berlin Heidelberg, Berlin, Heidelberg (2003). https://doi.org/10.1007/3-540-36494-3_54
7. Bouyer, P.: Forward analysis of updatable timed automata. *Formal Methods in System Design* **24**, 281–320 (2004). <https://doi.org/10.1023/B:FORM.0000026093.21513.31>
8. Bouyer, P., Gastin, P., Herbreteau, F., Sankur, O., Srivathsan, B.: Zone-based verification of timed automata: Extrapolations, simulations and what next? In: *Formal Modeling and Analysis of Timed Systems: 20th International Conference, FORMATS 2022, Warsaw, Poland, September 13–15, 2022, Proceedings*. p. 16–42. Springer-Verlag, Berlin, Heidelberg (2022). https://doi.org/10.1007/978-3-031-15839-1_2
9. Clarke, E.M., Henzinger, T.A., Veith, H., Bloem, R., et al.: *Handbook of model checking*, vol. 10. Springer (2018). <https://doi.org/10.1007/978-3-319-10575-8>
10. David, A., Larsen, K., Behrmann, G., Håkansson, J., Pettersson, P., Yi, W., Hendriks, M.: Uppaal 4.0. In: *Third International Conference on the Quantitative Evaluation of SysTems (QEST) 2006*. pp. 125–126. IEEE Computer Society Press, United States (2006). <https://doi.org/10.1109/QEST.2006.59>
11. Dill, D.L.: Timing assumptions and verification of finite-state concurrent systems. In: Sifakis, J. (ed.) *Automatic Verification Methods for Finite State*

- Systems. pp. 197–212. Springer Berlin Heidelberg, Berlin, Heidelberg (1990). https://doi.org/10.1007/3-540-52148-8_17
12. Gastin, P., Mukherjee, S., Srivathsan, B.: Fast algorithms for handling diagonal constraints in timed automata. In: Dillig, I., Tasiran, S. (eds.) *Computer Aided Verification*. pp. 41–59. Springer International Publishing, Cham (2019). https://doi.org/10.1007/978-3-030-25540-4_3
 13. Herbreteau, F., Srivathsan, B., Walukiewicz, I.: Better abstractions for timed automata. *Information and Computation* **251**, 67–90 (dec 2016). <https://doi.org/10.1016/j.ic.2016.07.004>
 14. Janowska, A., Janowski, P.: Slicing of timed automata with discrete data. *Fundam. Informaticae* **72**, 181–195 (2006)
 15. Liu, Y., Sun, J., Dong, J.S.: Developing model checkers using PAT. In: Bouajjani, A., Chin, W.N. (eds.) *Automated Technology for Verification and Analysis*. pp. 371–377. Springer Berlin Heidelberg, Berlin, Heidelberg (2010). https://doi.org/10.1007/978-3-642-15643-4_30
 16. Lund, S., van Diepen, J., Larsen, K.G., Muñoz, M., Jørgensen, T.R., Andersen, T.S.D.: An integer static analysis for better extrapolation in Uppaal. In: Dima, C., Shirmohammadi, M. (eds.) *Formal Modeling and Analysis of Timed Systems*. pp. 84–99. Springer International Publishing, Cham (2021). https://doi.org/10.1007/978-3-030-85037-1_6
 17. Nielson, F., Nielson, H.R., Hankin, C.: *Principles of Program Analysis*. Springer Publishing Company, Incorporated (2010). <https://doi.org/https://doi.org/10.1007/978-3-662-03811-6>
 18. Point, G., Herbreteau, F.: The TChecker tool and libraries, <https://github.com/ticktac-project/tchecker>
 19. Sipser, M.: *Introduction to the Theory of Computability* (2nd ed.). Computer Science Series, PWS Publishing Company (2006)